

Le Conservatoire National des Arts et Métiers

Rapport sur la reproductibilité d'un article scientifique sur le clustering

Fouille de données

30 mai 2026 par

Arsène MBABEH - Raphaël PÉNÉ - Oscar JOSEPH-GENESLAY

Promotion (2025-2026)

Titre : *Analyse et reproduction des méthodes de Clustering à partir de données simulées*

Table des matières

1	L'article	1
2	Importation des packages utilisés	1
3	Création d'une simulation de données	1
3.1	Génération des données	1
3.2	Visualisation des données - Avec MASS	2
3.3	Visualisation des données - Avec ggplot2	2
3.4	Visualisation en 3D de l'histogramme de la loi normale bivariée	3
3.5	Répartition des individus simulés	3
3.6	Visualisation des données en fonction de leur groupe	4
4	Création des variables discrètes	4
4.1	Génération des données discrètes	4
4.2	Visualisation de la fréquence des modalités	5
5	Le clustering	5
5.1	Application de la fonction k-means	5
5.2	Méthode du coude	6
6	Méthode des kmeans - algorithme d'Hartigan-Wong	7
6.1	L'algorithme d'Hartigan	7
6.2	Illustration avec 10 individus:	12
6.2.1	kmeans - Lloyd	15
6.2.2	Représentation de l'enveloppe convexe	16
6.3	Clustering Methods Based	18

1. L'article

Ce fichier **Quarto** utilise cet article Flynt and Dean (2016), l'objectif est de reproduire les résultats obtenus sur le sujet du clustering. Dans cette étude, on explore les méthodes de clustering en se basant sur des données simulées, en ayant pour objectif de reconstituer le clustering avec des méthodes statistiques.

2. Importation des packages utilisés

```
library(mvtnorm)
library(MASS)
library(dplyr)
library(ggplot2)
library(plot3D)
library(gplots)
library(tidyverse)
library(plotly)
library(tidyverse)
library(scales)
library(factoextra)
library(NbClust)
library(mclust)
library(kableExtra)
library(kStatistics)
```

3. Création d'une simulation de données

La génération des données simulées va permettre de tester les méthodes de clustering sur des données dont on connaît la répartition, sur lesquels on a nous même défini les groupes. On pourra ainsi comparer les groupes théoriques aux groupes trouvés avec les différentes fonctions.

3.1 Génération des données

Pour simuler les données, on définit une graine (seed) pour la reproductibilité.

On détermine le nombre d'individus dans chaque groupe. On doit avoir la répartition suivante :

- **Groupe 1** : 30%
- **Groupe 2** : 40%
- **Groupe 3** : 30%

Ensuite, on fixe les coordonnées des moyennes de chaque groupe, ce sont en fait les points moyens théoriques pour chaque groupe. Pour créer cela, on insère les coordonnées dans une matrice (2,3)

- **Groupe 1** : (0,0)
- **Groupe 2** : (3,5)
- **Groupe 3** : (0,6)

Une fois les centres des groupes définis, il faut déterminer comment les individus vont s'éparpiller autour de ces centres. C'est le rôle des matrices de covariance.

- **Sigma.sph = diag(c(1, 1))** : Cette ligne crée une matrice de dispersion en forme circulaire. Concrètement, les points s'écartent de la même manière dans toutes les directions.
- **Sigma.ell = matrix(...)** : Cette ligne crée une matrice de dispersion en forme elliptique. Les valeurs choisies font que le nuage de points sera étiré.

On génère aléatoirement le cluster théorique de 600 individus en fonction des spécificités de la répartition des groupes, c'est à dire que sur nos 600 individus, l'objectif de la fonction `sample()` est de générer 30% de 1 (Groupe 1), 40% de 2 (Groupe 2), 30% de 3 (Groupe 3).

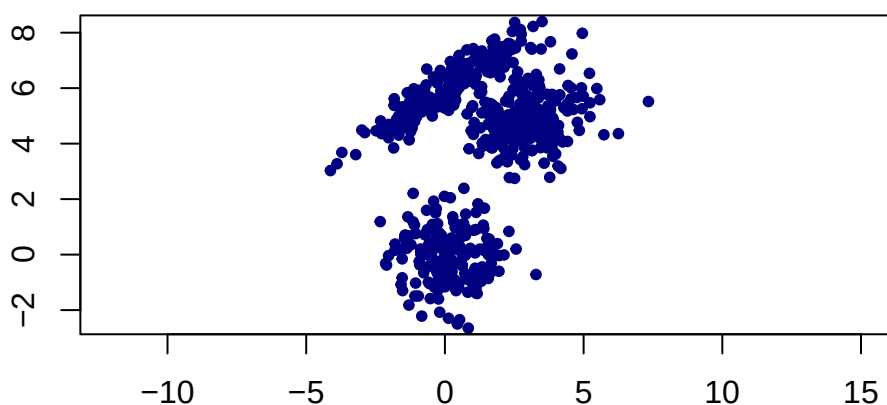
La dernière étape place les 600 individus sur le graphique, fonction de leur groupe :

- **Pour chaque individu (de 1 à 600)**, on regarde à quel groupe il appartient (`cl[i]`).
- On utilise la fonction `rmvnorm()` qui génère des points aléatoires selon une Loi Normale avec une condition `ifelse` :
 - **S'il est dans le groupe 3** : on lui donne les coordonnées du centre du groupe 3 et on lui applique la dispersion en forme d'ellipse.
 - **Sinon (s'il est dans le groupe 1 ou 2)** : on lui donne les coordonnées de son groupe et on lui applique la dispersion en forme circulaire.

3.2 Visualisation des données - Avec MASS

Voici la visualisation des données simulées. On distingue correctement les 3 groupes avec une forme elliptique et deux formes circulaires. Le package *MASS* met à disposition la fonction `eqscplot()` pour afficher le nuage de dispersion.

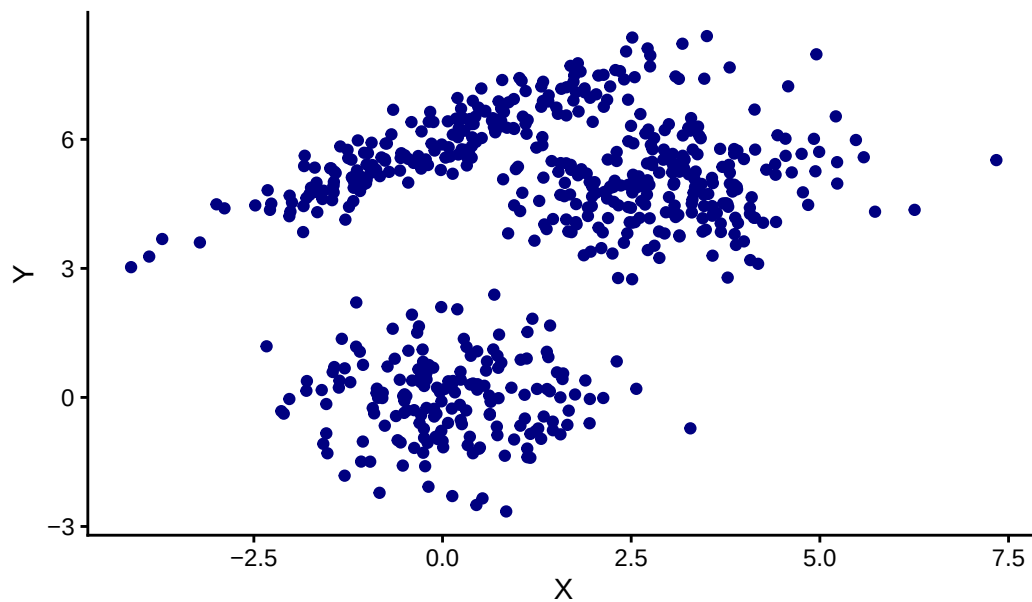
Répartition des points simulés dans un plan – MASS



3.3 Visualisation des données - Avec ggplot2

On peut aussi représenter les données en utilisant le package *ggplot2* et la méthode `geom_point()`

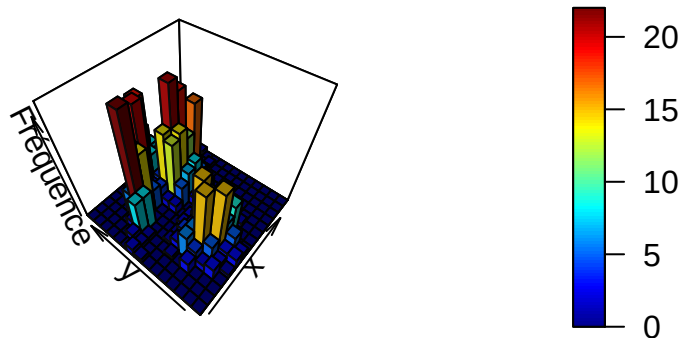
Répartition des points simulés dans un plan – ggplot2



3.4 Visualisation en 3D de l'histogramme de la loi normale bivariée

On peut représenter la loi normale bivariée des données à partir d'un plan en 3 dimensions. On retrouve bien nos 3 groupes. Les barres qui sont rouges sont les intervalles de données plus fréquents, ceux qui sont peu représentés sont plus en bleu. On est dans une loi normale bivariée, de paramètre (Espérance, Matrice variance-covariance).

Représentation 3D de la loi normale bivariée



3.5 Répartition des individus simulés

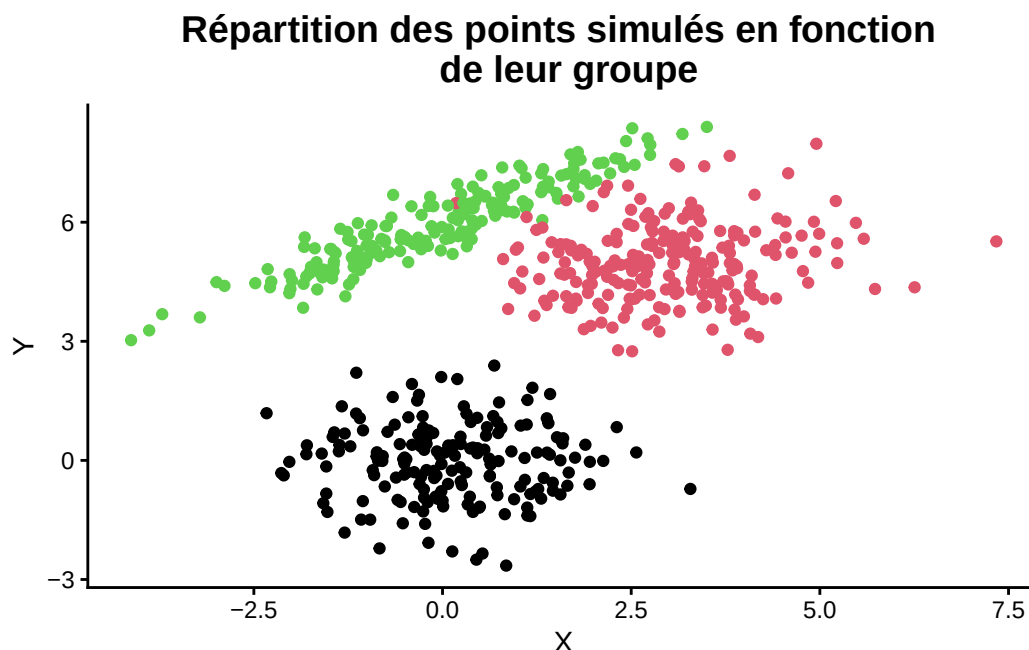
Le tableau ci-dessous représente les effectifs par groupe. Ils sont bien réparti comme nous l'avons configuré. Le groupe 1 est proche de 30%, le groupe 2 est proche de 40% et le groupe 3 est aussi proche de 30%. L'aléatoire et la seed configurés font que c'est pas parfaitement égal à notre

paramétrage.

	Groupe 1	Groupe 2	Groupe 3
Effectif	177 (29.5%)	237 (39.5%)	186 (31%)

3.6 Visualisation des données en fonction de leur groupe

On peut représenter visuellement les groupes théoriques des données simulées en attribuant une couleur en fonction de leur groupe. Désormais, on distingue plus facilement les 3 groupes, notamment la forme ellipsoïdale.



4. Création des variables discrètes

4.1 Génération des données discrètes

Cette partie recrée les quatre variables discrètes (trois binaires et une ternaire) qui accompagnent les variables continues dans le jeu de données simulé de l'article.

Voici le détail de la construction de chaque variable pour refléter la structure des groupes (où le groupe 1 correspond au groupe Noir, le groupe 2 au Rouge et le groupe 3 au Vert) :

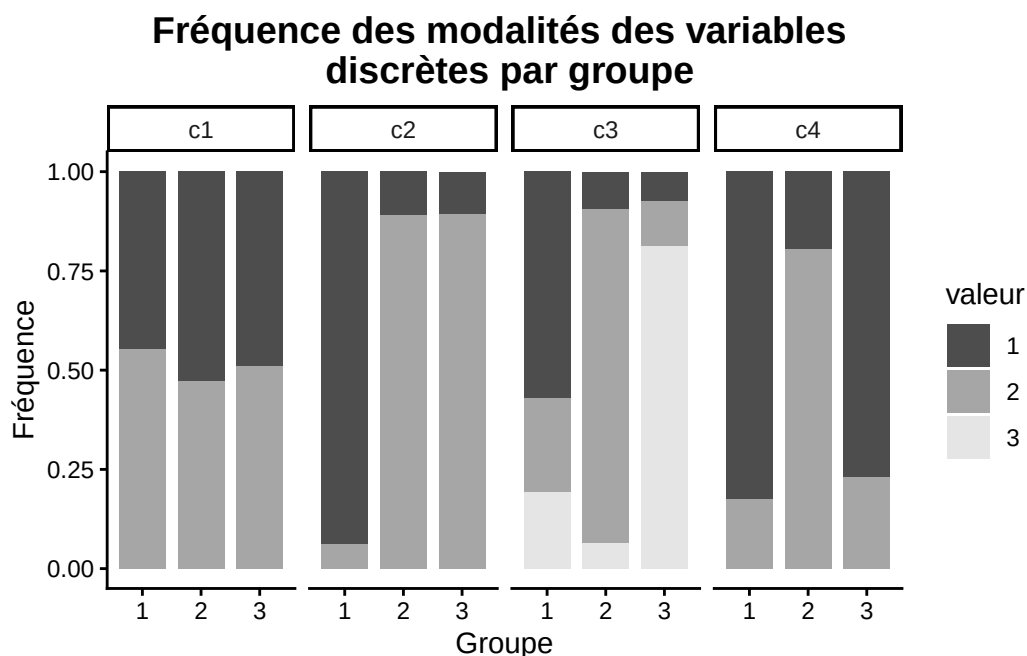
- Variable c1 : Elle est simulée avec une probabilité uniforme de 0.5 d'obtenir un succès (valeur 2) ou un échec (valeur 1), indépendamment du groupe d'appartenance de l'observation.
- Variable c2 : Il s'agit d'une variable binaire conçue pour avoir une probabilité de succès asymétrique. Le succès est beaucoup plus probable (0.9) pour les groupes 2 et 3 que pour le groupe 1 (0.1).
- Variable c3 : C'est la variable ternaire (modalités 1, 2 et 3) de l'étude. Sa distribution est paramétrée pour qu'une observation ait une plus forte probabilité d'obtenir un 1 si elle est dans le groupe noir, un 2 dans le groupe rouge et un 3 dans le groupe vert.

- Variable c4 : C'est une variable binaire dont la probabilité de succès a été configurée pour être nettement supérieure dans le groupe rouge (probabilité de 0.8).

		c1	c2	c3	c4
1.6137555	4.6784541	1	2	2	2
0.3135524	-0.3026124	2	1	2	1
3.1383011	3.7670414	2	2	2	1
-1.5400580	-0.1531943	1	1	1	1
2.5165331	4.8541686	1	2	2	1
1.9779610	4.4491698	1	2	2	2

4.2 Visualisation de la fréquence des modalités

Nous représentons les variables discrètes avec le graphique à barres suivant, pour distinguer visuellement la proportion des modalités des variables discrètes par groupe.



5. Le clustering

5.1 Application de la fonction k-means

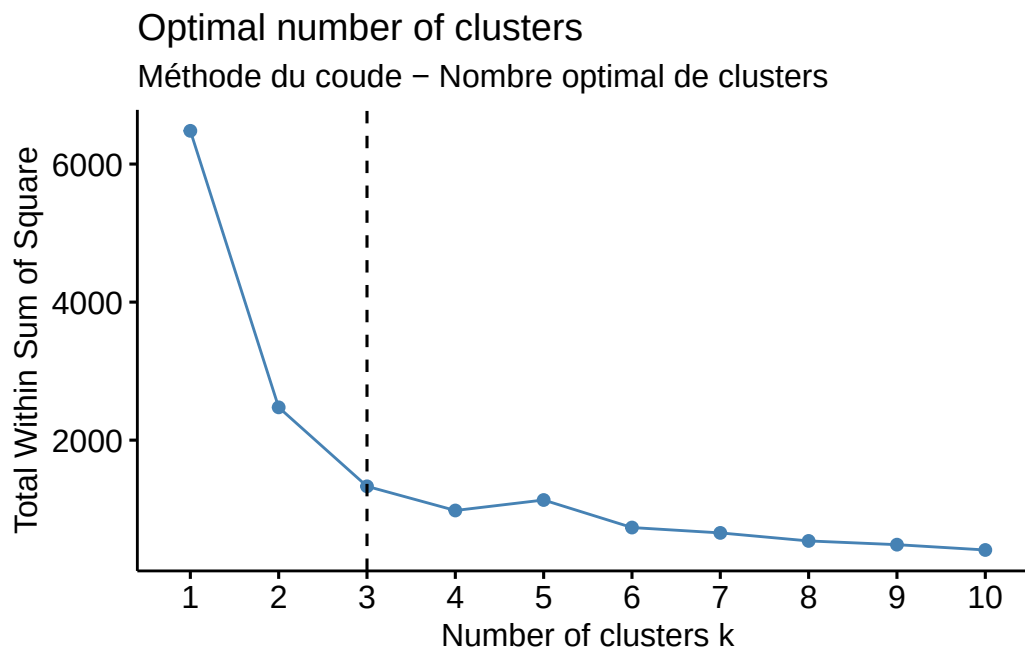
Nous aurions pu explorer toutes les possibilités pour 2 clusters une par une à la main, mais c'est impossible vu le nombre de possibilités pour 600 observations. Il faudrait effectuer la démarche le nombre de fois ci-dessous :

[1] 2.074758e+180

5.2 Méthode du coude

La méthode du coude sert définir visuellement le nombre optimal de clusters afin de conserver le plus d'information possible. Nous devons nous arrêter au moment où l'inertie ne chute plus de manière significative.

Dans le graphique ci-dessous, le coude commence à se former à partir du cluster $k = 3$. Nous conservons donc que 3 clusters.

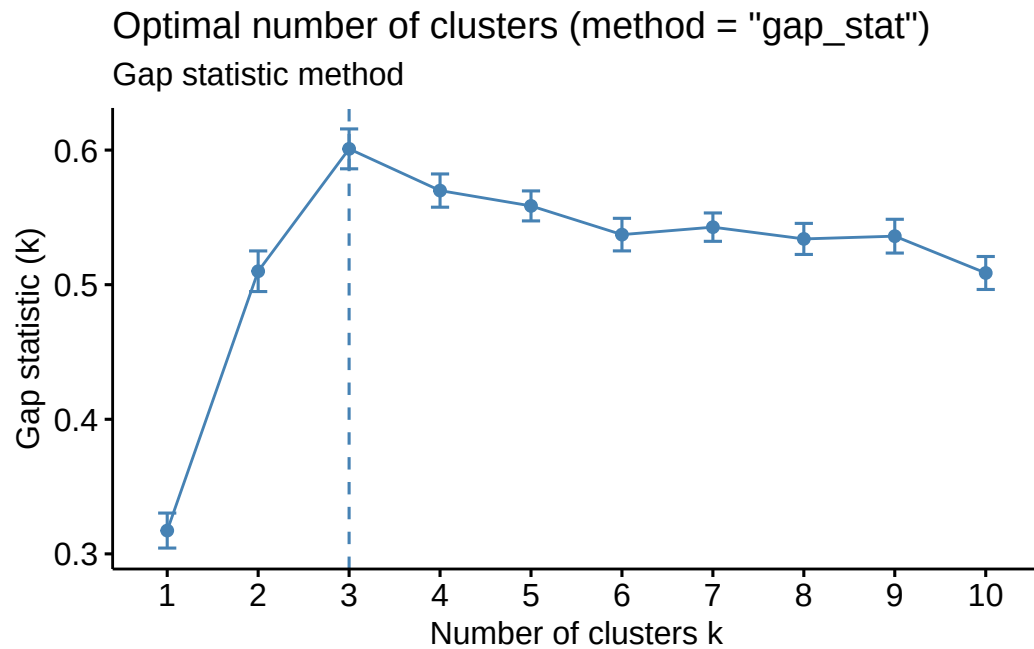


La méthode du coude est performante pour trouver le nombre de cluster optimal mais reste très dépendante de notre interprétation.

Dans la méthode ci-dessous, la méthode “Gap statistic”, le nombre optimal de clusters k est celui qui donne le Gap le plus grand. C’est une méthode plus mathématique et souvent plus robuste que la méthode du coude visuelle.

Elle compare la dispersion intra-cluster observée dans les données à l’espérance mathématique de cette dispersion générée sous une distribution uniforme de référence (hypothèse nulle d’absence de clusters). Le k optimal est la valeur qui maximise cet écart.

Avec cette méthode, on détecte automatiquement que le nombre de clusters optimal est $k = 3$.



6. Méthode des kmeans - algorithme d'Hartigan-Wong

6.1 L'algorithme d'Hartigan

Description de l'algorithme d'Hartigan-Wong pour la méthode des kmeans.

Algorithme de Hartigan-Wong (K-Means)

1. **Initialisation** : Distribuer chaque point dans un cluster au hasard.
2. **Calcul des centres** : Calculer le centre moyen de chaque groupe formé.
3. **Boucle principale** : Pour chaque point du jeu de données :
 4. **Test de transfert** : Calculer si déplacer le point vers un autre groupe rend l'ensemble des clusters plus compact.
 5. **Décision** : Si le transfert améliore la qualité globale :
 6. **Action** : Déplacer le point dans le nouveau groupe immédiatement.
 7. **Mise à jour** : Recalculer aussitôt le centre du groupe qui a perdu le point et celui du groupe qui l'a gagné.
8. **Convergence** : Recommencer l'étape 3 tant que des points bougent encore.

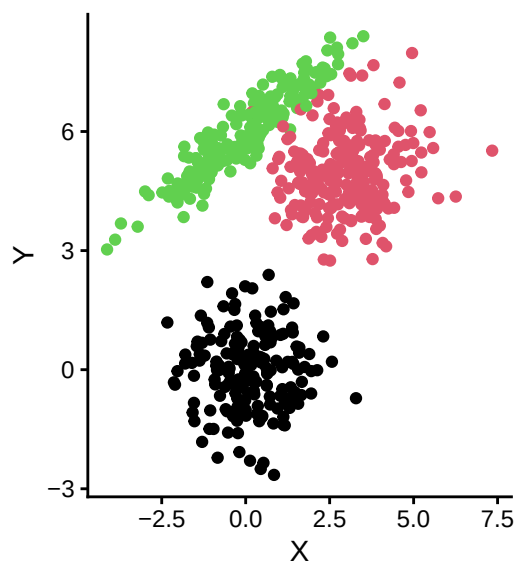
```
library(gridExtra)
cluster1 = kmeans (vars [,1:2], 1, nstart = 50)
cluster2 = kmeans (vars [,1:2], 2, nstart = 50) # dans k1 il y a 181 individus / dans k2 il y en a
cluster3 = kmeans (vars [,1:2], 3, nstart = 50)

library(tidyverse)
# Point moyen : moyenne_1 & moyenne_2
as_tibble(vars[,1:2]) %>% summarise(moyenne_1 = mean(V1),
                                   moyenne_2 = mean(V2),
                                   Variance = (var(V1) + var(V2)) * 599 # Somme des distances euclidiennes p
                                   )
```

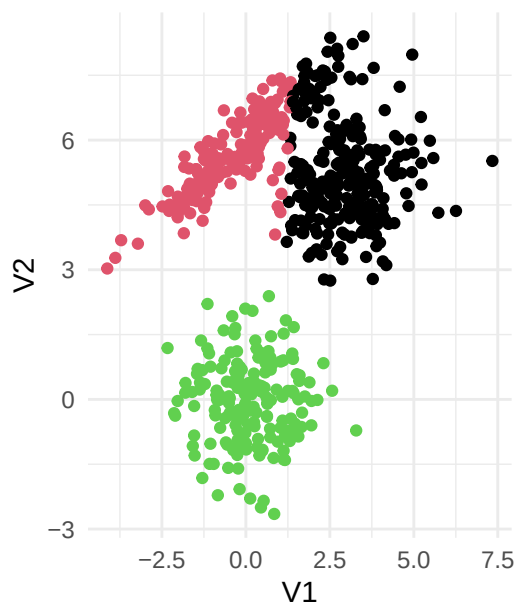
```
# A tibble: 1 x 3
  moyenne_1 moyenne_2 Variance
  <dbl>     <dbl>     <dbl>
1     1.18       3.79    6481.
```

```
as_tibble(X) %>% ggplot(aes(x = V1, y=V2)) + geom_point(color = cluster3$cluster) + theme_minimal()
grid.arrange(graph1, graph2, nrow = 1)
```

artition des points simulés en
de leur groupe



Représentation des variables



```
cl
```

```
[1] 2 1 2 1 2 2 3 2 2 3 1 3 1 2 3 3 2 2 2 3 3 1 2 1 2 2 2 2 1 1 2 1 1 3 2 1
[38] 1 2 1 2 3 2 1 2 3 1 2 1 3 3 1 2 3 1 1 2 3 2 2 1 1 3 3 3 2 3 2 1 1 3 2 2 3
[75] 1 3 2 1 2 2 2 2 1 1 2 3 3 1 2 1 3 3 1 3 1 1 2 2 2 3 2 1 2 3 2 2 3 2 1 3 2
[112] 2 2 3 1 1 3 2 2 3 1 2 2 1 2 3 3 2 1 2 3 2 3 1 2 2 2 2 1 3 2 3 1 2 2 3 1 1
[149] 3 3 3 2 3 1 3 1 3 2 3 3 3 3 2 3 2 3 2 1 2 3 3 1 1 1 1 2 1 2 1 2 3 2 1 2 2
[186] 2 1 3 2 2 3 1 2 2 2 2 3 1 3 3 2 3 2 1 3 1 2 2 2 2 2 1 1 3 2 3 3 3 3 2 3 3
[223] 2 3 3 3 3 2 2 2 2 1 1 2 2 3 3 3 3 2 2 2 2 3 3 2 1 3 1 3 2 3 3 3 3 3 2 2 1
[260] 2 3 1 1 2 3 2 1 1 1 2 1 2 2 1 1 2 3 1 3 1 2 1 3 3 2 2 2 2 3 2 2 3 3 2 1 1
[297] 3 2 3 1 1 2 3 1 1 1 1 3 2 1 2 2 3 2 1 1 3 2 2 2 2 3 1 3 1 2 2 3 2 2 2 1 1
[334] 3 1 1 2 2 2 2 2 2 3 1 3 2 3 2 3 3 2 3 3 2 2 1 2 1 3 3 1 1 1 1 1 2 1 2 3 3
[371] 2 3 1 2 2 3 2 2 2 1 2 1 1 3 2 1 1 3 3 3 3 2 2 2 1 2 1 2 2 3 2 3 1 1 3 2 3
[408] 2 2 3 1 1 1 1 2 3 1 3 3 2 1 3 1 2 1 3 2 3 1 1 2 1 2 3 3 3 1 2 2 2 2 3 2 2
[445] 3 2 3 2 3 2 2 3 2 2 1 1 1 3 2 2 1 1 1 1 1 3 1 2 2 1 3 2 1 3 2 2 3 1 1 3
[482] 3 2 3 3 3 2 2 3 3 1 2 2 3 2 1 1 3 1 3 1 2 3 1 2 3 2 2 2 3 1 2 3 1 3 2 2 1
[519] 2 2 3 2 1 1 1 3 2 1 1 1 2 2 2 2 1 1 3 2 2 1 3 2 1 3 3 1 3 2 1 3 2 3 3 2 3
[556] 1 2 3 2 2 2 2 2 1 1 1 3 2 1 3 3 3 1 2 1 3 3 3 3 1 1 1 1 2 1 2 3 2 1 3 2 3
[593] 2 3 1 1 1 1 2 2
```

```
cluster3$cluster
```

```
[1] 1 3 1 3 1 1 2 1 1 2 3 2 3 1 2 2 1 1 1 1 2 2 3 1 3 1 1 1 1 3 3 2 3 3 1 1 3
[38] 3 1 3 1 2 1 3 1 2 3 1 3 1 2 3 1 1 3 3 1 2 1 1 3 3 2 2 2 2 2 1 3 3 2 1 1 2
[75] 3 2 2 3 2 1 1 1 3 3 1 2 2 3 1 3 2 2 3 2 3 3 1 1 1 2 1 3 1 2 1 1 1 1 3 2 1
[112] 1 1 2 3 3 2 1 1 2 3 1 1 3 1 2 2 1 3 1 2 1 2 3 1 1 1 1 3 2 1 1 3 1 1 2 3 3
[149] 2 2 2 1 2 3 2 3 2 1 2 2 2 2 1 2 1 2 1 3 1 2 2 3 3 3 3 1 3 2 3 1 2 1 3 1 1
[186] 1 3 2 1 1 2 3 1 1 1 1 2 3 2 1 1 1 1 3 2 3 1 1 1 1 1 3 3 2 1 1 1 2 1 1 1 2
[223] 1 1 1 2 2 1 1 1 1 3 3 1 1 2 2 2 2 1 1 1 1 2 2 1 3 2 3 2 1 2 2 2 1 1 1 1 3
[260] 1 2 3 3 1 1 2 3 3 3 1 3 1 1 3 3 1 2 3 2 3 1 3 2 2 1 1 1 1 2 1 1 2 2 1 3 3
[297] 2 1 1 3 3 1 2 3 3 3 3 2 1 3 1 1 1 1 3 3 2 1 1 1 1 2 3 2 3 1 1 2 1 1 2 3 3
[334] 2 3 3 1 1 1 1 1 1 2 3 1 1 2 1 2 2 1 2 2 1 1 3 1 3 1 2 3 3 3 3 3 1 3 1 2 2
[371] 1 2 3 1 1 2 1 1 1 3 1 3 3 2 1 3 3 1 2 2 2 1 1 1 3 1 3 1 1 1 1 2 3 3 2 1 2
[408] 1 1 2 3 3 3 3 1 2 3 2 2 1 3 2 3 1 3 2 1 2 3 3 1 3 1 2 2 2 3 1 1 1 1 2 1 1
[445] 2 1 2 1 1 1 1 1 1 3 3 3 1 2 1 3 3 3 3 3 3 2 3 1 1 3 2 1 3 1 1 1 1 3 3 1
[482] 2 1 2 1 1 1 1 2 2 3 1 1 2 1 3 3 2 3 2 3 1 2 3 1 2 1 1 2 1 3 1 2 3 2 1 1 3
[519] 1 1 2 1 3 3 3 2 1 3 3 3 1 1 1 1 3 3 2 1 1 3 2 1 3 2 2 3 2 1 3 2 1 2 2 1 2
[556] 3 1 2 1 1 1 1 1 3 3 3 2 1 3 2 2 2 3 1 3 2 2 2 2 3 3 3 3 2 3 1 2 1 3 2 1 1
[593] 1 1 3 3 3 3 1 1
```

```
table(tibble(cl, cluster3$cluster))
```

```
      cluster3$cluster
cl      1      2      3
1      0      0 177
2    227    10      0
3     33   153      0
```

```
library(clue)
solve_correspondance = solve_LSAP(table(tibble(cl, cluster3$cluster)), maximum = TRUE)

conf_matrice = table(tibble(cl, cluster3$cluster))[, solve_correspondance]

1-sum(diag(conf_matrice))/600 # le taux de points mal placés
```

```
[1] 0.07166667
```

L'article aurait dû utiliser l'indice de Rand pour évaluer la concordance entre les données simulées et la répartition trouvée par `kmeans()`. Explicatoin du fonctionnement du Rand : L'indice de Rand est une mesure de similarité entre deux partitions d'un ensemble. Il est principalement utilisé en catégorisation automatique. Son principe est de mesurer la consistance (le taux d'accord) entre deux partitions.

Prenons un exemple pour illustrer la méthode, supposons une classe qui a 9 élèves, on veut comparer les deux partitions suivante : - partition 1 : {A,B}, {C,D,E,F,G}, {H,I} - partition 2 : {A}, {B}, {C,D,E}, {F,G}, {H,I}

L'indice va compter le nombre de paires parmi les 36 qui sont soit dans un même groupe, soit dans deux groupes différents dans la partition 1 et la partition 2.

Dans cet exemple, on trouve 80,6% de concordance.

```
library(fossil)
```

Loading required package: sp

Loading required package: maps

Attaching package: 'maps'

The following object is masked from 'package:mclust':

map

The following object is masked from 'package:purrr':

map

Loading required package: shapefiles

Loading required package: foreign

Attaching package: 'shapefiles'

The following objects are masked from 'package:foreign':

read.dbf, write.dbf

```
g1 = c(1, 1, rep(2, 5), 3, 3)
g2 = c(1, 2, 3, 3, 3, 4, 4, 5, 5)

rand.index(g1, g2)
```

```
[1] 0.8055556
```

```
(7+7+6+6+6+5+5+8+8)/36/2
```

```
[1] 0.8055556
```

Application de l'indice de Rand à nos données de clustering. Le kmeans() avec $k = 3$ nous donne de concordance de 90,1%.

```
rand.index(cl, cluster3$cluster)
```

```
[1] 0.9090707
```

La classification hiérarchique :

méthode single linkage

```
dist(vars[, 1:2], diag = TRUE, method = "euclidean")-> distance
```

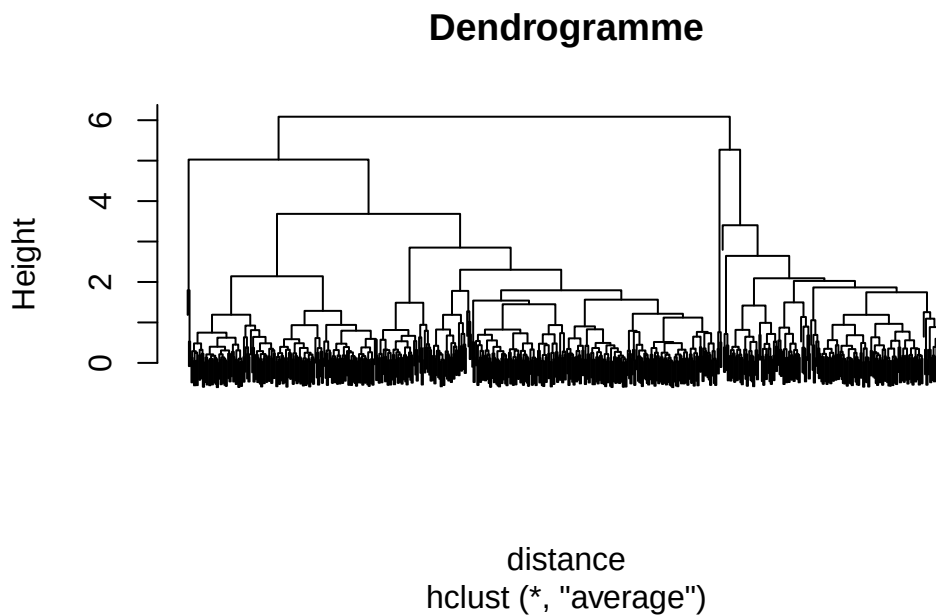
```
library(fastcluster)
```

Attaching package: 'fastcluster'

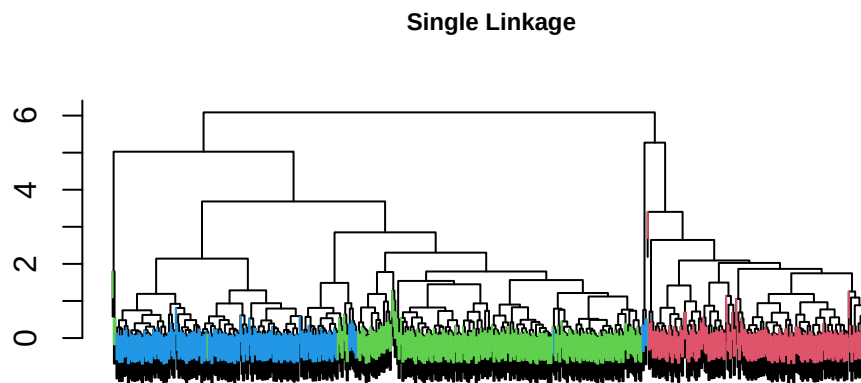
The following object is masked from 'package:stats':

hclust

```
arbre <- hclust(distance, method = "average")  
plot(arbre, labels = FALSE, main = "Dendrogramme")
```



```
library(sparcl)  
ColorDendrogram(arbre, y = cl, main="Single Linkage", xlab = "", sub = "", cex = .75)
```



```
rand.index(c1, cutree(arbre, k=3))
```

```
[1] 0.7559154
```

```
solve_correspondance_arbre = solve_LSAP(table(tibble(c1, cutree(arbre, k=3))), maximum = TRUE)
```

```
conf_matrice_arbre = table(tibble(c1, cutree(arbre, k=3))[, solve_correspondance_arbre])
conf_matrice_arbre
```

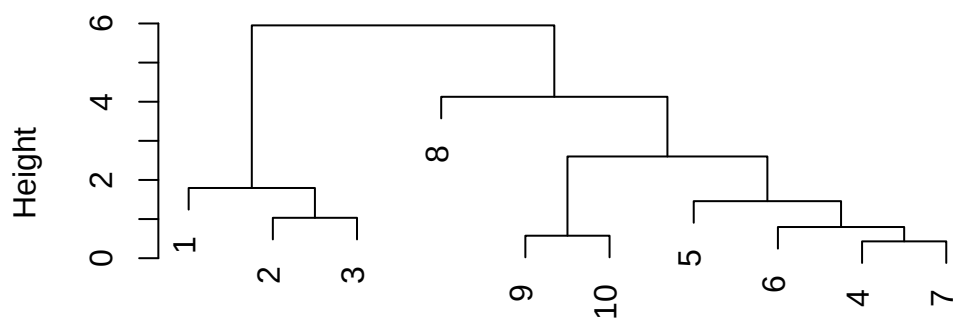
```
cutree(arbre, k = 3)
c1      2      1      3
1 177      0      0
2      0 237      0
3      0 182      4
```

6.2 Illustration avec 10 individus:

```
cbind(X, c1) -> data_10
```

```
data_10 %>% as_tibble() %>% arrange(c1) %>% slice(1:3, 178:181, 598:600)-> data_10
cluster_10 = kmeans (data_10, 3, nstart = 50)
dist(data_10[,1:2], diag = TRUE, method = "euclidean")-> distance_10
arbre_10 <- hclust(distance_10, method = "average")
plot(arbre_10, main = "Dendrogramme")
```

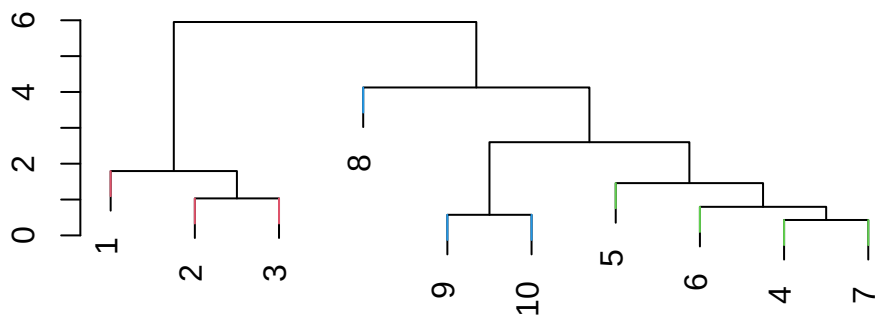
Dendrogramme



distance_10
hclust (*, "average")

```
ColorDendrogram(arbre_10, y = data_10$c1, main="Single Linkage", xlab = "", sub = "", cex = .75, l
```

Single Linkage



```
data_10 %>% as.data.frame() %>% ggplot(aes(x = V1, y = V2, label = row.names(data_10)))+  
  geom_point(aes(color = factor(data_10$c1)), size = 2.5)+geom_text(hjust = 2)+  
  theme_minimal() -> graph_10_2
```

```
cbind(graph_10_1, graph_10_2)
```

```
graph_10_2  
[1,] <ggplot2::ggplot>
```

```
library(dendextend)
```

```
-----
```

Welcome to dendextend version 1.19.1

Type citation('dendextend') for how to cite the package.

Type browseVignettes(package = 'dendextend') for the package vignette.

The github page is: <https://github.com/talgalili/dendextend/>

Suggestions and bug-reports can be submitted at: <https://github.com/talgalili/dendextend/issues>

You may ask questions at stackoverflow, use the r and dendextend tags:

<https://stackoverflow.com/questions/tagged/dendextend>

To suppress this message use: `suppressPackageStartupMessages(library(dendextend))`

```
-----
```

Attaching package: 'dendextend'

The following object is masked from 'package:stats':

cutree

```
library(circlize)
```

```
=====
```

circlize version 0.4.18

CRAN page: <https://cran.r-project.org/package=circlize>

Github page: <https://github.com/jokergoo/circlize>

Documentation: https://jokergoo.github.io/circlize_book/book/

If you use it in published research, please cite:

Gu, Z. circlize implements and enhances circular visualization
in R. Bioinformatics 2014.

This message can be suppressed by:

`suppressPackageStartupMessages(library(circlize))`

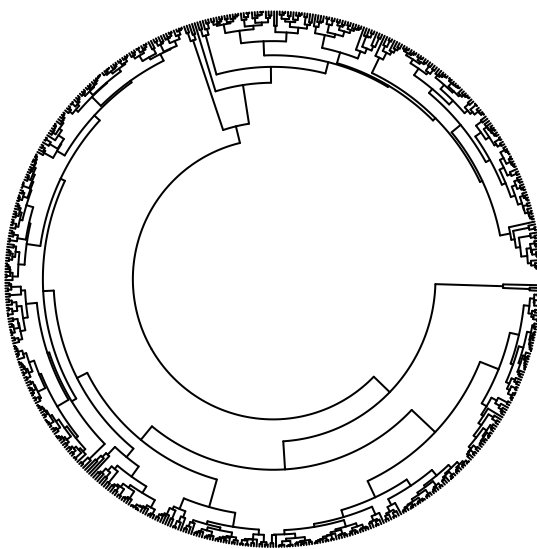
```
=====
```

```
# Hierarchical clustering dendrogram
```

```
hc <- as.dendrogram(arbre)
```

```
# Circular dendrogram
```

```
circlize_dendrogram(hc,
                     labels_track_height = NA,
                     dend_track_height = 0.5,
                     labels = FALSE)
```

6.2.1 *kmeans* - Lloyd

Algo de Lloyd

Algorithme de Lloyd (K-Means)

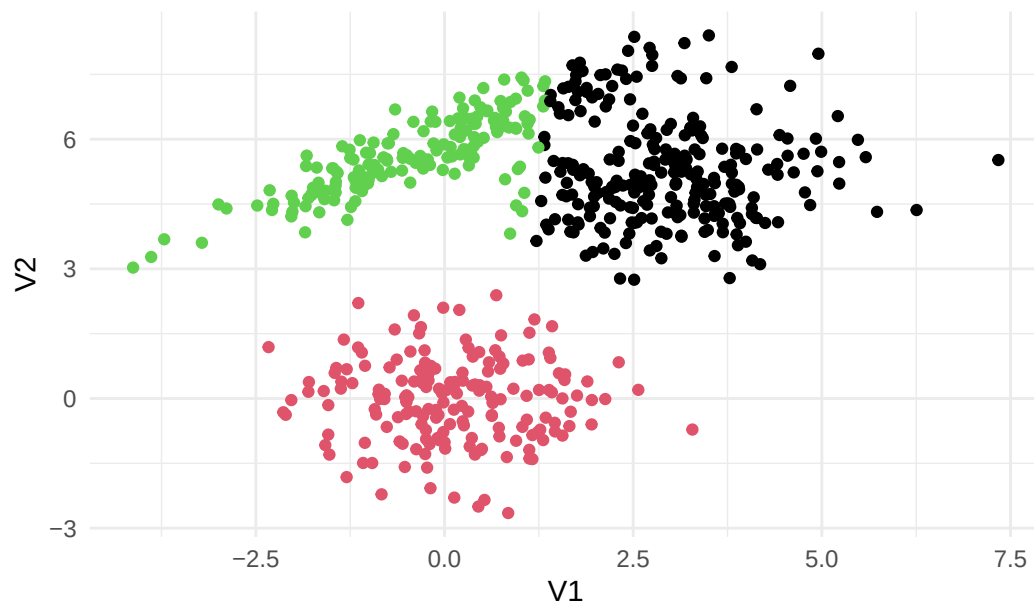
1. **Initialisation** : Choisir k points au hasard pour être les centres.
2. **Boucle principale : Répéter tant qu'il y a des changements** :
 3. **Assignment globale** : Chaque point choisit le centre le plus proche sans se soucier des autres points.
 4. **Attente** : On attend que TOUS les points aient choisi leur groupe.
 5. **Mise à jour groupée** : Une fois que tout le monde a fini de bouger, on recalcule tous les nouveaux centres en une seule fois.
6. **Convergence** : Arrêter quand les centres ne bougent plus d'une étape à l'autre.

```
cluster3_lloyd = kmeans (vars [,1:2], 3, nstart = 50, algorithm = "Lloyd")
```

```
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
Warning: did not converge in 10 iterations
```

```
as_tibble(X) %>% ggplot(aes(x = V1, y=V2)) + geom_point(color = cluster3_lloyd$cluster) + theme_mi
```

Représentation des variables continues en fonction des groupe



ESSAYER L'AUTRE METHODE

6.2.2 Représentation de l'enveloppe convexe

```
# 1. Combiner vos données, la couleur d'origine (cl) et les résultats du k-means
df <- as_tibble(X) %>%
  mutate(
    Couleur_Origine = cl,
    # On extrait les affectations aux clusters de l'objet kmeans en facteur
    Cluster_Kmeans = as.factor(cluster3_lloyd$cluster)
  )

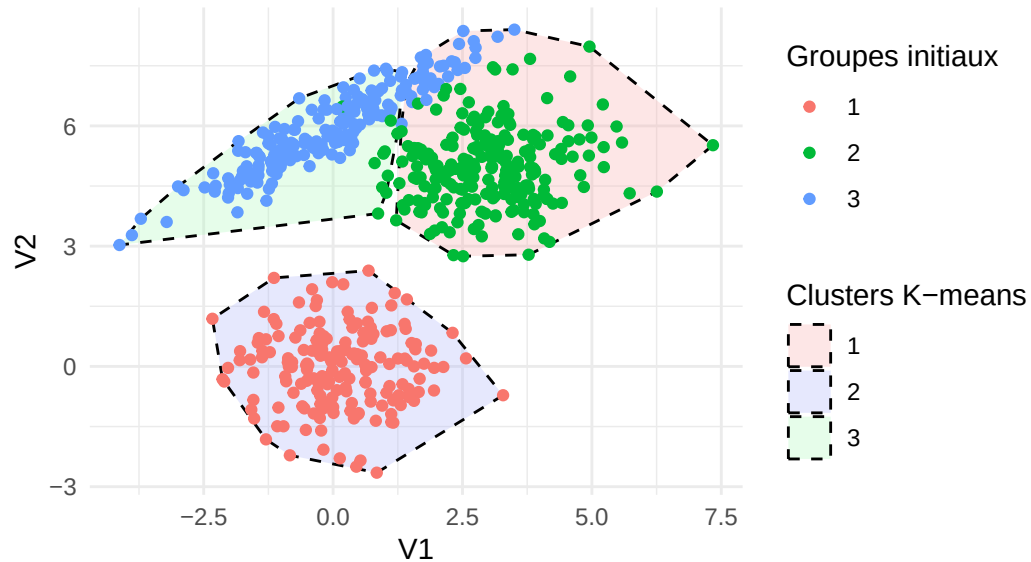
# 2. Calculer les points formant l'enveloppe convexe pour CHAQUE cluster k-means
# La fonction chull() renvoie les indices des points qui forment la bordure convexe
hulls <- df %>%
  group_by(Cluster_Kmeans) %>%
  slice(chull(V1, V2))

# 3. Créer le graphique combiné
ggplot(df, aes(x = V1, y = V2)) +
  # Étape A : Tracer les enveloppes convexes (polygones) des K-means
  # alpha = 0.3 permet de rendre la surface transparente pour voir les points
  geom_polygon(data = hulls,
    aes(fill = Cluster_Kmeans, group = Cluster_Kmeans),
    alpha = 0.3, color = "black", linetype = "dashed") +
  scale_fill_manual(values = c("#FAADAD", "#ADB4FA", "#ADFABA"))+

  # Étape B : Tracer les points avec leurs couleurs d'origine (groupes prédéfinis)
  geom_point(aes(color = factor(Couleur_Origine))) +
```

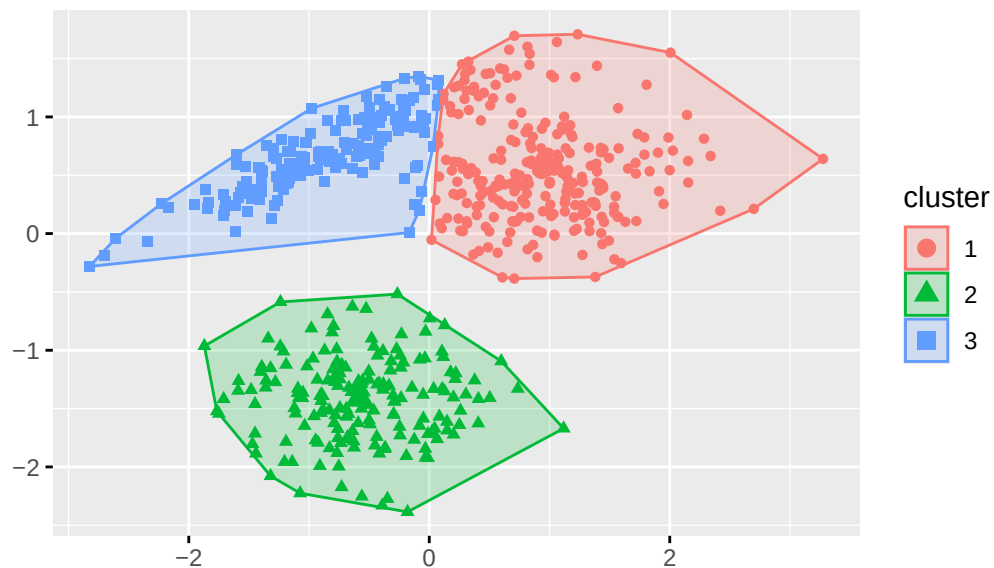
```
# Esthétique
theme_minimal() +
ggtitle("Représentation des variables continues en fonction des groupes prédéfinis\net enveloppes
labs(fill = "Clusters K-means", color = "Groupes initiaux")
```

Représentation des variables continues en fonction des groupe
et enveloppes convexes des K-means



```
fviz_cluster(cluster3_lloyd,
  data = vars[, 1:2],
  ellipse.type = "convex",
  geom = "point", # "point" évite d'afficher le numéro de chaque ligne
  show.clust.cent = TRUE, # Affiche le centre des clusters (croix)
  main = "Représentation des clusters K-means")
```

Représentation des clusters K-means



6.3 Clustering Methods Based

EXPLIQUER LE PRINCIPE

on est dans le cas multivariée

C'est le maximum de vraisemblance, avec le BIC le plus grand

Après on compare la simulation avec cette méthode

```
#Fit the mbc model for a range of number of components, default 1 to 9 and all variance parameters
m1 = Mclust(vars[,1:2])
m1
```

```
'Mclust' model object: (VVE,3)
```

Available components:

[1]	"call"	"data"	"modelName"	"n"
[5]	"d"	"G"	"BIC"	"loglik"
[9]	"df"	"bic"	"icl"	"hypvol"
[13]	"parameters"	"z"	"classification"	"uncertainty"

```
#'Mclust' model object:
# best model: ellipsoidal, equal orientation (VVE) with 3 components

#Look at the summary of the model
summary(m1)
```

```
-----
Gaussian finite mixture model fitted by EM algorithm
-----
```

Mclust VVE (ellipsoidal, equal orientation) model with 3 components:

log-likelihood	n	df	BIC	ICL
-2230.692	600	15	-4557.338	-4572.94

Clustering table:

1	2	3
229	177	194

```
#m1$parameters
```

```
#-----
# Gaussian finite mixture model fitted by EM algorithm
#-----

# Mclust VVE (ellipsoidal, equal orientation) model with 3 components:

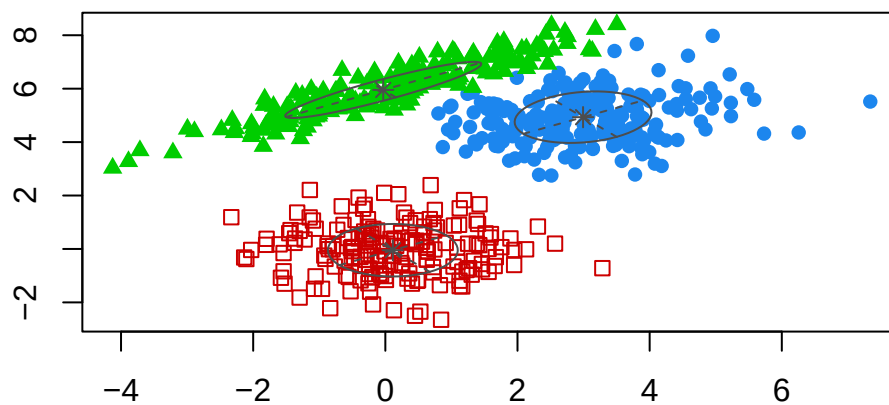
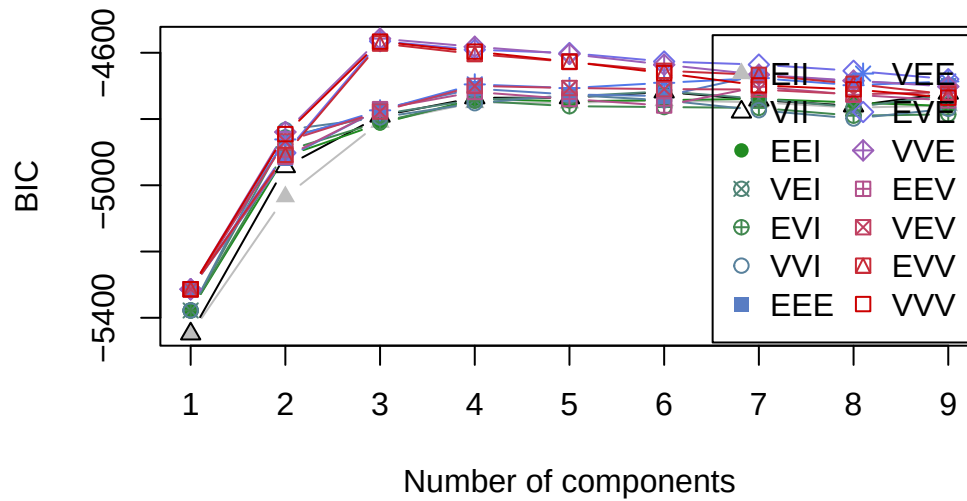
# log.likelihood    n df          BIC          ICL
#-2230.702 600 15 -4557.359 -4572.325
```

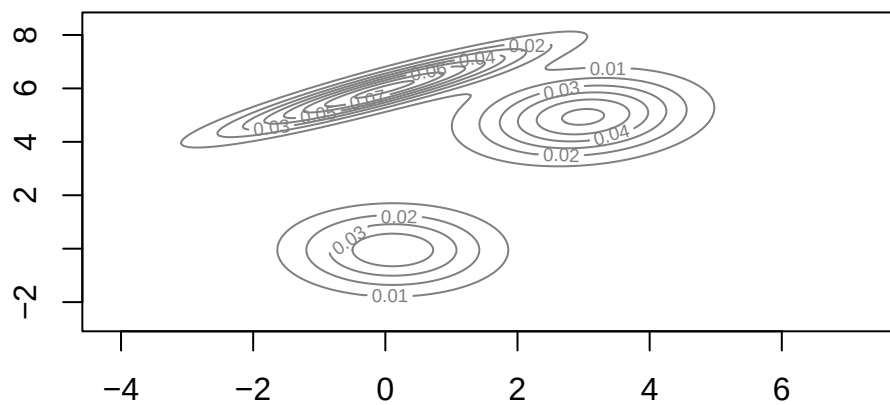
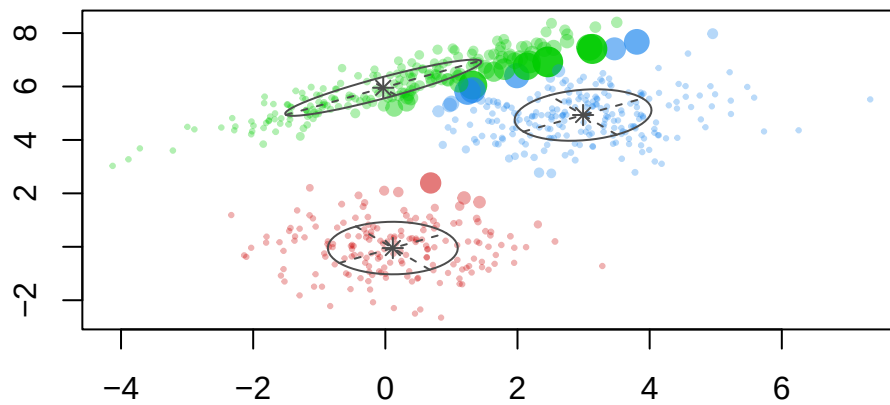
```
#Clustering table:
```

```
# 1 2 3
#229 177 194
```

```
#Look at the BIC, classification and other plots for this model
```

```
plot(m1)
```





```
# cross-classification table  
#table(cl, m1$class)
```

References

Flynt, A., & Dean, N. (2016). A survey of popular r packages for cluster analysis. *Journal of Educational and Behavioral Statistics*, 41(2), 205–225.